Project Deployment & Implementation Manual

LoRaWAN Smart Metering Pipeline for NILM (Appliance Fingerprinting)

Team: Robin Singh & Abhigyan Kesari

1 System Architecture Overview

This system captures high-resolution appliance power signatures at the edge and securely routes them over a LoRaWAN network to a local environment for storage and visualization. The foundational architecture consists of three core components:

1. **The Edge (Hardware):** An STM32 Nucleo-WL55JC microcontroller equipped with an X-CUBE-PM33A1 metrology shield to sample real-time electrical parameters (V_{rms}, I_{rms}, Active Power, and Watt-hours).
2. **The Network (LoRaWAN):** Data is transmitted as a tight, 10-byte hex payload to a local gateway, which routes it to The Things Network (TTN) console.
3. **The Infrastructure Backend:** A Python-based MQTT bridge acts as an intermediary listener, downloading data from TTN and pushing it into a database, while an analytics platform renders the user interface.

—

2 Core Infrastructure Software Explanations

2.1 What is Docker?

Instead of manually installing database engines, configuring ports, dealing with operating system dependencies, and matching library versions, the backend infrastructure utilizes **Docker**.

Docker is a containerization platform that packages software environments into isolated units called "containers." By reading the included `docker-compose.yml` file, Docker downloads, installs, and links a pre-configured TimescaleDB instance and a Grafana instance with a single command. This ensures identical runtime behavior across different host machines.

2.2 What is Grafana?

Grafana is an open-source data visualization and analytics platform. It connects directly to time-series databases (like TimescaleDB) and allows users to build dynamic dashboards with charts, gauges, and alerts. In this project, Grafana serves as our primary engineering

command center to monitor live electrical waveforms and visualize distinct appliance power signatures.

—

3 Key Project Files

The repository layout contains the following critical files required for evaluation:

- **Edge Firmware Location:**
  - `STM32Cube_FW_WL_V1.4.0/Projects/NUCLEO-WL55JC/Applications/LoRaWAN/LoRaWAN_End_Node`
    Contains the required files to run the metering and data transmission. This folder must be placed in the original repository directory.
- **Backend Deployment Location:**
  - `ttn_bridge.py`: The Python MQTT data engine that catches incoming uplinks, extracts values, and handles the database writes.
  - `docker-compose.yml`: Defines the automated stack layout for the TimescaleDB database and Grafana containers.

—

4 Step-by-Step Installation & Deployment Guide

4.1 Prerequisite: Installing Docker

Before running the project backend, the host machine must have Docker installed:

- **Windows / macOS:** Download and install **Docker Desktop** from the official website (`https://www.docker.com/products/docker-desktop/`). Ensure virtualization is enabled in your system's BIOS settings.
- **Linux (Ubuntu/Debian):** Run the following commands in the terminal to clear old installations and configure the official Docker engine:

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io
   docker-compose-plugin
```

4.2 Step 1: Configuring The Things Network (TTN)

Before the local backend can intercept data, the hardware must be registered on the TTN cloud console to establish a secure handshake and decode the raw radio packets.

1. Log in to the TTN Console, select your cluster, and create or open your designated Application.
2. Click **Register end device**. Enter the exact `DevEUI`, `JoinEUI`, and `AppKey` that are flashed onto the Nucleo board to enable Over-The-Air Activation (OTAA).

3. Once the device is registered, navigate to the **Payload formatters** tab and select **Uplink**.

4. Change the formatter type to **Custom Javascript** and paste the following decoding script. This logic perfectly slices the 10-byte hex payload back into accurate floating-point electrical metrics:

```
function decodeUplink(input) {
  var bytes = input.bytes;

  if (bytes.length !== 10) {
    return { errors: ["Invalid payload length. Expected 10 bytes
       ."] };
  }

  var voltage = ((bytes[0] << 8) | bytes[1]) / 100.0;
  var current = ((bytes[2] << 8) | bytes[3]) / 1000.0;
  var active_power = ((bytes[4] << 8) | bytes[5]) / 10.0;
  var apparent_power = ((bytes[6] << 8) | bytes[7]) / 10.0;
  var energy_raw = ((bytes[8] << 8) | bytes[9]);

  return {
    data: {
      voltage_V: voltage,
      current_A: current,
      power_active_W: active_power,
      power_apparent_VA: apparent_power,
      energy_accumulator_Wh: energy_raw
    }
  };
}
```

4.3 Step 2: Instantiating the Backend Infrastructure

Navigate to the project root folder (`smart-meter-ml`) containing your `docker-compose.yml` file using a terminal or command prompt, and run:

```
docker compose up -d
```

Note: The first execution will automatically fetch and compile the database and visualization image layers from the cloud registry. The `-d` flag forces the stack to run smoothly in the background.

4.4 Step 3: Starting the Python MQTT Engine

With the core containers awake, execute the data bridge script to listen for the incoming LoRa packets from TTN:

```
python ttn_bridge.py
```

Upon successful execution, the terminal will report connection verification and log real-time data drops as they hit the database.

4.5 Step 4: Accessing and Authenticating Grafana

1. Launch an internet browser and connect to: `http://localhost:3000`
2. At the login portal, use the default administrative access tokens:
   - **Username:** `admin`
   - **Password:** `admin` (Grafana will prompt you to update this upon initialization).

4.6 Step 5: Connecting the TimescaleDB Data Source

Because this is a clean, unconfigured installation, Grafana must be manually pointed to the database container:

1. In the left-hand navigation pane, click on the **Gear Icon (Connections)** → **Data Sources**.
2. Click **Add data source** and select **PostgreSQL** from the catalog (TimescaleDB is built directly on top of PostgreSQL).
3. Configure the internal networking access credentials exactly as mapped out below:
   - **Host:** `timescaledb:5432`
   - **Database Name:** `meter_db`
   - **User:** `postgres`
   - **Password:** `meterpass123`
   - **SSL Mode:** `disable`
4. Scroll down to the bottom and click **Save & Test**. Grafana should confirm connection stability with a green validation alert.

4.7 Step 6: Generating a Visualization Panel from Scratch

To populate an active visualization chart monitoring real-time active power:

1. Click the **Plus (+) Icon** in the top-right menu bar → Select **Dashboard** → Click **Add a new panel**.
2. In the query control center at the bottom of the window, switch from the standard visual builder to the **Edit SQL** view.
3. Input the structured time-series fetch query to access the active data tables:

```
SELECT
  time AS "time",
  active_power AS "Active␣Power␣(W)"
FROM power_metrics
WHERE
  $__timeFilter(time)
ORDER BY time ASC;
```

4. In the right-hand layout configurations panel, set the panel title to `"Real-Time Active Power Waveform"` and select **Time Series** as the graph option.

5. Click **Apply** in the top-right corner to securely dock the panel onto your live operational workspace dashboard.

—

5 Key Technical Implementation Details

During early development, evaluating the native hardware-level 64-bit energy accumulation registers exposed severe data corruption risks when transmitted across constrained sub-GHz industrial networks. Background electromagnetic current fluctuations regularly triggered phantom power artifacts within the lower bit fields.

Our Engineering Resolution: We chose to bypass the raw hardware storage architectures entirely. We integrated a resilient, software-driven energy integration module inside `lora_app.c`. By intercepting accurate operational power signatures ($ActPow$) alongside active millisecond time tracking loops (`HAL_GetTick()`), we successfully isolated the pipeline from floating ambient noise. This allows clean, aggregated power trends to be accurately calculated down to 16-bit payload variables without losing precision.